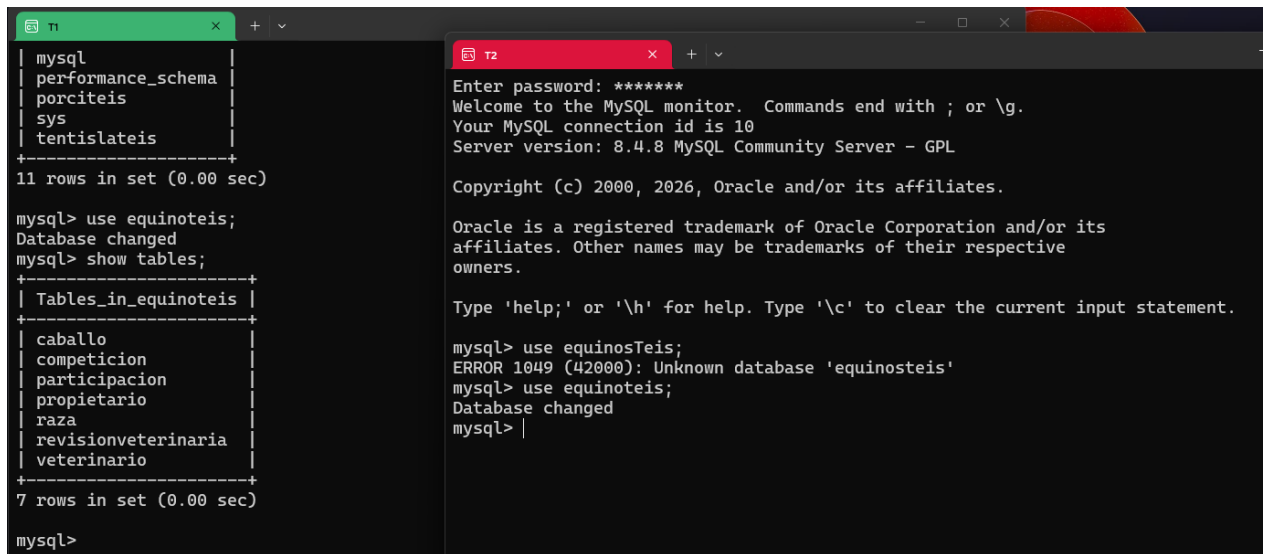


CONEXIÓN CON LA BBDD



The image shows two terminal windows side-by-side. The left window, titled 'T1', shows a MySQL prompt where the user has entered a password and is now in the 'equinoteis' database. They have run 'show tables;' and the output lists 7 tables: caballo, competicion, participacion, propietario, raza, revisionveterinaria, and veterinario. The right window, titled 'T2', shows the MySQL monitor's welcome message, including the connection ID (10) and server version (8.4.8). It also shows an error message 'ERROR 1049 (42000): Unknown database 'equinosteis'' when the user tried to use a non-existent database, followed by a successful switch to the 'equinoteis' database.

```
T1
mysql
| performance_schema |
| porcisteis         |
| sys                |
| tentislateis       |
+-----+
11 rows in set (0.00 sec)

mysql> use equinoteis;
Database changed
mysql> show tables;
+-----+
| Tables_in_equinoteis |
+-----+
| caballo              |
| competicion          |
| participacion         |
| propietario          |
| raza                 |
| revisionveterinaria  |
| veterinario          |
+-----+
7 rows in set (0.00 sec)

mysql>

T2
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.4.8 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

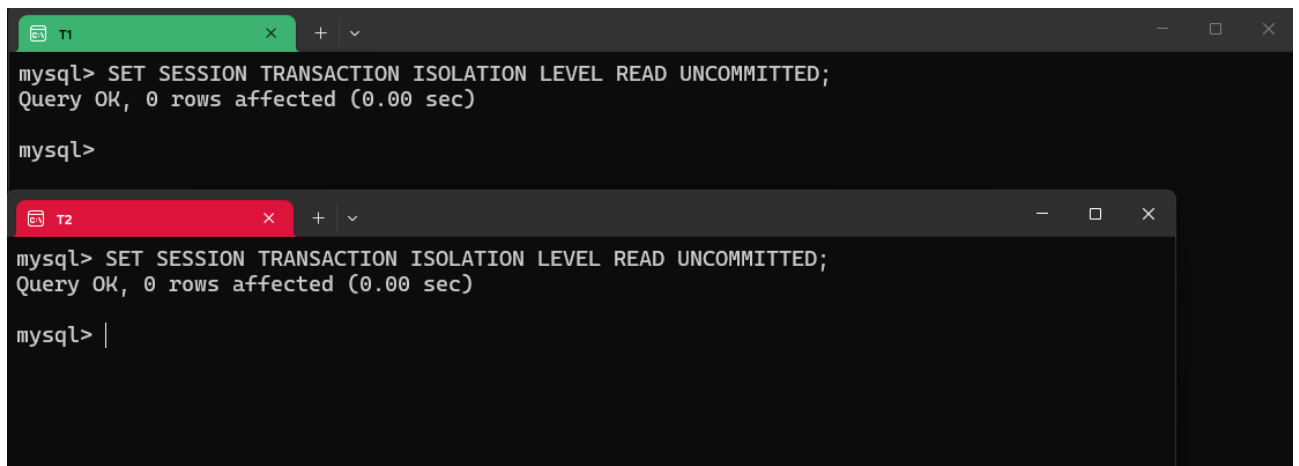
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use equinosTeis;
ERROR 1049 (42000): Unknown database 'equinosteis'
mysql> use equinoteis;
Database changed
mysql>
```

PREPARACIÓN INICIAL

Definimos el nivel de aislamiento que vamos a usar

- SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;



The image shows two terminal windows side-by-side. Both windows show the same command being executed: 'SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;'. The output for both is 'Query OK, 0 rows affected (0.00 sec)'. The left window is titled 'T1' and the right window is titled 'T2'.

```
T1
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql>

T2
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

FENÓMENO 1: Dirty Read

```
T1
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Caballo SET estado_salud='En Observación' WHERE id_caballo=1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT estado_salud FROM Caballo WHERE id_caballo=1;

T2
mysql> SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT estado_salud FROM Caballo WHERE id_caballo=1;
+-----+
| estado_salud |
+-----+
| En Observación |
+-----+
1 row in set (0.00 sec)

mysql> |
```

```
T1
mysql> UPDATE Caballo SET estado_salud='En Observación' WHERE id_caballo=1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT estado_salud FROM Caballo WHERE id_caballo=1;
+-----+
| estado_salud |
+-----+
| En Observación |
+-----+
1 row in set (0.00 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT estado_salud FROM Caballo WHERE id_caballo=1;
+-----+
| estado_salud |
+-----+
| Bueno |
+-----+
1 row in set (0.00 sec)

mysql>

T2
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT estado_salud FROM Caballo WHERE id_caballo=1;
+-----+
| estado_salud |
+-----+
| En Observación |
+-----+
1 row in set (0.00 sec)

mysql> SELECT estado_salud FROM Caballo WHERE id_caballo=1;
+-----+
| estado_salud |
+-----+
| Bueno |
+-----+
1 row in set (0.00 sec)

mysql>
```

Qué pasa: T2 lee el estado 'En Observación', pero luego T1 cancela todo (ROLLBACK). T2 ha leído un dato "fantasma" que nunca se guardó.

¿Es lo esperado? Sí. Al estar en READ UNCOMMITTED, el gestor permite leer datos modificados en memoria, aunque no tengan COMMIT. Es la definición exacta de este fenómeno.

FENÓMENO 2: Dirty Write (Escritura Sucia)

- Orden de Comandos:
 - o T1. Start Transaction;
 - o T2. Start Transaction;
 - o T1. UPDATE Caballo SET estado_salud='Cuidado' WHERE id_caballo=4;
 - o UPDATE Caballo SET estado_salud='Excelente' WHERE id_caballo=4; (**T2 se quedará congelada**)

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Caballo SET estado_salud='Cuidado' WHERE id_caballo=4;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql>
```

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Caballo SET estado_salud='Excelente' WHERE id_caballo=4;
|
```

- o T1. COMMIT; (**T2 se descongelas**)
- o T2. COMMIT;

Qué pasa: T2 se queda bloqueada esperando a que T1 termine. No logramos sobrecribir el dato a la vez.

¿Es lo esperado? No exactamente. Un nivel de aislamiento bajo como *READ UNCOMMITTED* debería permitir la máxima flexibilidad en la lectura de datos, incluso aceptando inconsistencias temporales. Sin embargo, en los motores como MySQL, incorporan mecanismos de protección que no dependan únicamente del nivel de aislamiento.

FENÓMENO 3: Non-repeatable Read (Lectura no repetible)

- Orden de Comandos:
 - o T1. START TRANSACTION;
 - o T1. SELECT peso FROM Caballo WHERE id_caballo = 2;

```
T1
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo = 2;
+-----+
| peso  |
+-----+
| 510.00 |
+-----+
1 row in set (0.00 sec)

mysql>
```

- o T2. START TRANSACTION;
- o T2. UPDATE Caballo SET peso = 580 WHERE id_caballo=2;
- o T1. SELECT peso FROM Caballo WHERE id_caballo = 2;

```
T1
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo = 2;
+-----+
| peso  |
+-----+
| 510.00 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo = 2;
+-----+
| peso  |
+-----+
| 580.00 |
+-----+
1 row in set (0.00 sec)

mysql>

T2
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Caballo SET peso = 580 WHERE id_caballo=2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> |
```

- o T2. COMMIT;
- o T1. COMMIT;

```
T1
+-----+
| 510.00 |
+-----+
1 row in set (0.00 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo = 2;
+-----+
| peso |
+-----+
| 580.00 |
+-----+
1 row in set (0.00 sec)

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo = 2;
+-----+
| peso |
+-----+
| 580.00 |
+-----+
1 row in set (0.00 sec)

mysql>

T2
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> UPDATE Caballo SET peso = 580 WHERE id_caballo=2;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo = 2;
+-----+
| peso |
+-----+
| 580.00 |
+-----+
1 row in set (0.00 sec)

mysql>
```

Qué pasa: T1 lee dos pesos diferentes en la misma transacción. De hecho, el dato cambia antes incluso de que T2 guarde los cambios.

¿Es lo esperado? Sí, pero en una forma más extrema. Al trabajar sin bloqueos de lectura, una transacción no solo puede ver cambios confirmados por otras, sino también modificaciones aún no consolidadas. Como resultado, **cualquier alteración realizada por otra transacción impacta de inmediato en los datos que estamos consultando.**

FENÓMENO 4: Lost Update (Actualización Perdida)

- Orden de Comandos:
 - T1: START TRANSACTION;
 - T2: START TRANSACTION;
 - T1: SELECT peso FROM Caballo WHERE id_caballo=3; (Ambos leen, ej. 600)
 - T2: SELECT peso FROM Caballo WHERE id_caballo=3; (Ambos leen 600)
 - T1: UPDATE Caballo SET peso=610 WHERE id_caballo=3;
 - T2: UPDATE Caballo SET peso=620 WHERE id_caballo=3; (T2 se congela)
 - T1: COMMIT (**Se descongela T2**)

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo=3;
+-----+
| peso  |
+-----+
| 620.00 |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE Caballo SET peso=610 WHERE id_caballo=3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql>

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo=3;
+-----+
| peso  |
+-----+
| 620.00 |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE Caballo SET peso=620 WHERE id_caballo=3;
Query OK, 1 row affected (7.83 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> |
```

- T2: COMMIT;

Qué pasa: El trabajo de T1 (610) desaparece. T2 lo aplasta con su 620 porque leyó el dato inicial al mismo tiempo.

```
mysql> SELECT peso FROM Caballo WHERE id_caballo=3;
+-----+
| peso  |
+-----+
| 620.00 |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE Caballo SET peso=610 WHERE id_caballo=3;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT peso FROM Caballo WHERE id_caballo=3;
+-----+
| peso  |
+-----+
| 620.00 |
+-----+
1 row in set (0.00 sec)

mysql> |
```

¿Es lo esperado? Sí. READ UNCOMMITTED no protege contra este fallo lógico. T2 se bloqueó por seguridad física (como vimos en el fenómeno 2), pero al descongelarse, aplicó su actualización basada en información obsoleta, machacando el trabajo de T1.

FENÓMENO 5: Phantom Read (Lectura Fantasma)

- Orden de Comandos:
 - T1: START TRANSACTION;
 - T1: SELECT * FROM Caballo WHERE altura >= 1.70;
 - T2: START TRANSACTION;
 - T2: INSERT INTO Caballo VALUES (99,'Pegaso',DATE '2020-01-01','Macho','Blanco',500,1.75,'Excelente',1,2);
 - T1: SELECT * FROM Caballo WHERE altura >= 1.70;

```
mysql> Query OK, 0 rows affected (0.00 sec)
mysql> SELECT * FROM Caballo WHERE altura >= 1.70;
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id_caballo | nombre | fecha_nacimiento | sexo | color | peso | altura | estado_salud | id_raza | id_propietario |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 3 | Furia | 2014-01-10 | Macho | Marrón | 610.00 | 1.72 | Bueno | 3 | 3 |
| 5 | Titán | 2016-11-11 | Macho | Negro | 900.00 | 1.82 | Excelente | 5 | 4 |
| 13 | Fénix | 2014-03-03 | Macho | Negro | 520.00 | 1.70 | Excelente | 6 | 2 |
| 17 | Huracán | 2012-02-02 | Macho | Negro | 610.00 | 1.74 | Excelente | 3 | 3 |
| 21 | Fulgor | 2019-04-01 | Macho | Alazán | 520.00 | 1.71 | Bueno | 1 | 2 |
+----+-----+-----+-----+-----+-----+-----+-----+
5 rows in set (0.00 sec)

mysql> SELECT * FROM Caballo WHERE altura >= 1.70;
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| id_caballo | nombre | fecha_nacimiento | sexo | color | peso | altura | estado_salud | id_raza | id_propietario |
+----+-----+-----+-----+-----+-----+-----+-----+-----+
| 3 | Furia | 2014-01-10 | Macho | Marrón | 610.00 | 1.72 | Bueno | 3 | 3 |
| 5 | Titán | 2016-11-11 | Macho | Negro | 900.00 | 1.82 | Excelente | 5 | 4 |
| 13 | Fénix | 2014-03-03 | Macho | Negro | 520.00 | 1.70 | Excelente | 6 | 2 |
| 17 | Huracán | 2012-02-02 | Macho | Negro | 610.00 | 1.74 | Excelente | 3 | 3 |
| 21 | Fulgor | 2019-04-01 | Macho | Alazán | 520.00 | 1.71 | Bueno | 1 | 2 |
| 99 | Pegaso | 2020-01-01 | Macho | Blanco | 500.00 | 1.75 | Excelente | 1 | 2 |
+----+-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

mysql>

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO Caballo VALUES (99,'Pegaso',DATE '2020-01-01','Macho','Blanco',500,1.75,'Excelente',1,2);
Query OK, 1 row affected (0.01 sec)

mysql>
```

- T2: ROLLBACK; (Para limpiar el registro y no ensuciar la BBDD)
- T1: COMMIT;

Qué pasa: Aparecen filas nuevas en una consulta repetida.

¿Es lo esperado? SÍ. Al no haber bloqueos de rango (Range Locks), otras transacciones pueden insertar datos libremente. Además, como estamos en READ UNCOMMITTED, T1 ve al caballo fantasma "Pegaso" incluso antes de que T2 haga COMMIT.

FENÓMENO 6: Write Skew (Sesgo de Escritura)

- Orden de Comandos:
 - o T1: START TRANSACTION;
 - o T2: START TRANSACTION;
 - o T1: SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
 - o T2: SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
+-----+
| COUNT(*) |
+-----+
|      9   |
+-----+
1 row in set (0.00 sec)

mysql> |

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
+-----+
| COUNT(*) |
+-----+
|      9   |
+-----+
1 row in set (0.00 sec)

mysql> |
```

- o T1: UPDATE Caballo SET estado_salud='Bueno' WHERE id_caballo=5;
- o T2: UPDATE Caballo SET estado_salud='Bueno' WHERE id_caballo=8;
- o T1: COMMIT;
- o T2: COMMIT;

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
+-----+
| COUNT(*) |
+-----+
|      9   |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE Caballo SET estado_salud='Bueno' WHERE id_caballo=5;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
+-----+
| COUNT(*) |
+-----+
|      7   |
+-----+
1 row in set (0.00 sec)

mysql> |

mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
+-----+
| COUNT(*) |
+-----+
|      9   |
+-----+
1 row in set (0.00 sec)

mysql> UPDATE Caballo SET estado_salud='Bueno' WHERE id_caballo=8;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> SELECT COUNT(*) FROM Caballo WHERE estado_salud='Excelente';
+-----+
| COUNT(*) |
+-----+
|      7   |
+-----+
1 row in set (0.00 sec)

mysql> |
```

Resultado esperado: Se observa el fenómeno de **Write Skew**. Ambas transacciones leen que hay suficientes caballos excelentes (7) para permitirse "degradar" a uno cada una. Como modifican filas distintas, el sistema no detecta conflicto físico y permite que ambas guarden los cambios.

¿Es lo que debería pasar? Sí desde el punto de vista físico, pero no desde el lógico.

En un nivel de aislamiento como *READ UNCOMMITTED*, prácticamente no existe separación entre transacciones. Esto permite que múltiples procesos operen simultáneamente. Para evitar este comportamiento sería necesario un nivel de aislamiento como *SERIALIZABLE*, donde las transacciones se ejecutan de forma efectiva una tras otra